

Developing an automated data pipeline for cryptocurrency: Ethereum, and a system for detecting statistical anomalies

Chonlathit Phuncam
phuncam.c@gmail.com
<https://linktr.ee/kyo.skykim>
<https://www.linkedin.com/in/chonlathitp>

Abstract

The high volatility of the cryptocurrency market necessitates continuous monitoring and analysis. This research project designed and developed an end-to-end automated data pipeline for Ethereum (ETH), utilizing a Python architecture to automatically retrieve data via the CoinGecko API. Pyspark and Spark SQL are used in Databricks for data transformation and processing. Financial indicators, specifically the standard Z-score, are calculated to systematically detect price anomalies. The collected data is then presented on a Databricks dashboard. The pipeline successfully detected ETH price trends during high-risk market periods when the absolute Z-score more than 2. Ultimately, this research demonstrates the integration of Data Engineering, Data Analysis, and financial data analytics to create a reliable structure supporting data-driven investment decision-making.

Keyword : PySpark, SQL, Z-score, Data visualization

Acknowledgments

This independent research project document is prepared for further study and to develop practical skills in the fields of Data Engineering and Data Visualization.

I would like to express my gratitude to DataSpark TH for providing the learning and training opportunities, which have helped me understand the workflow of a Data Engineer and enable me to develop myself in a tangible way. I also thank Passawee Wanwilai and Weerapat Panyatheera for their advice and initiative in undertaking this project. Finally, I extend my thanks to Sirasek Trachovanich, Tanakorn Palakul, Pannatat Channoparat, Nopphanat Khongkham, Parintorn Kewsuwan, Thawin Thanapronsuksan, Prin Dispat, Thawin Lawanwisut, Phurinat Kaewsima, Sorakrit Duangsin, Nitchakun Saengrungrueang, Thanamas Thanasorn, Sirawich Roopngam, and many other contributors. Thank you for the encouragement and the exchange of perspectives, which helped my ideas solidify and make this project even more complete.

1 Introduction

Crypto currencies have fundamentally transformed the financial landscape. Starting with the launch of Bitcoin as a peer-to-peer[3] electronic cash system, and building upon that, Ethereum has grown to become the world's second-largest cryptocurrency[7]. Despite their rapid adoption, digital assets are characterized by extremely high price volatility and inherent market imperfections[6]. These factors pose significant challenges for market participants and underscore the need for advanced analytical frameworks to effectively understand and manage these risks. Financial time series analysis[5] is widely used to assess asset price behavior. However, the high-frequency nature of cryptocurrency data makes manual tracking impossible, necessitating the implementation of automated data processing using powerful tools like Databricks. By automating market data retrieval via the CoinGecko API, data can be processed efficiently and reliably. This research focuses on developing a comprehensive automated data processing system specifically for Ethereum. The system will evaluate key indicators such as daily returns and market volatility and utilize statistical methodologies such as Z-scores to systematically detect price anomalies. The final results will be integrated into a comprehensive dashboard, bridging the gap between raw data processing and actionable financial insights.

2 Literature Review

Time-series analysis is used in financial markets to study the behavior of asset prices and risk. It analyzes return and volatility as fundamental parameters, using the standard deviation over a given time period to reflect market risk levels. Furthermore, the Z-score is used to identify events indicating price changes relative to the normal trend in cryptocurrency. This is done using tools capable of handling large datasets and processing data efficiently, such as Apache Spark.

3 Tools and management

The development of an improved data management tool pipeline ensures reliable data retrieval, verification, and visualization using the following tools.

- Databricks serves as the primary platform for this project. It's a Software as a Service (SaaS) built with the Lakehouse architecture, designed to run on the cloud, and is widely recognized as a highly efficient tool for managing large datasets.
- Python is the foundational and primary language used to design and develop pipeline systems, extracting raw ETH data from the CoinGecko API.
- Pyspark is used on Databricks to transform data into a clear and clean data format. The workflow demonstrates the ETL (Extract, Transform, Loading) method for handling time series data.
- Spark SQL is used to query complex data and calculate business indicators using high-level SQL, such as Windows, efficiently calculating daily returns, means, standard deviations, and Z-cores.

- The Databricks dashboard is primarily used in research to present data. The results are then connected to the Databricks dashboard, allowing users to quickly view price trends, volatility, and detect anomalies[2].

4 Research methodology

This research employs a quantitative data analysis approach, including data collection, processing, and statistical indicator calculations. Ethereum price data is retrieved from the CoinGecko API[1], providing time-series data. This data is then processed using PySpark to improve data management efficiency.

Therefore, we obtained interesting indicators to analyze, as follows:

1) Daily Return

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

where

r_t Rate of return on date t ,

P_t Closing price of the asset on date t ,

P_{t-1} Closing price of the asset on the previous day.

This indicator is used to measure the change in price from one day to another in percentage terms.

2) Volatility

Calculated from the standard deviation of the rate of return over a given period.

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (r_i - \bar{r})^2}$$

Where

σ is the volatility,

r_i is the daily rate of return,

$\bar{r}$ is the average rate of return,

n is the number of days in the time period used for calculation.

Disclaimer: High volatility indicates a higher level of market risk.

3) Z-score for anomaly detection

$$z = \frac{x - \mu}{\sigma}$$

Where

z is the Z-score,

x is the asset price on the date under consideration,
 μ is the rolling mean of the price over the given period,
 σ is the standard deviation of the price over the same period.

The Z-score is used to measure how much the current price deviates from the average. If $|z| > 2$, it is considered an anomaly.

All these indicators are used together to analyze trends, risks, and key events in the market.

5 Information Data Architecture

Designing a comprehensive data pipeline to support processes from data retrieval to analysis. The process begins with data retrieval via API, with error handling and connection timeouts to enhance system stability. Data is then processed using PySpark, with a clearly defined schema to prevent data inaccuracies. Once the data is cleaned, it is stored in a tabular format and analyzed using SQL to calculate various metrics. Finally, the results are presented via a dashboard for easy user interpretation. This structure provides a flexible, easy-to-maintain, and scalable system.

6 Results

The analysis results demonstrate the price behavior of Ethereum. The developed data pipeline is practical and capable of statistical analysis. The studied data shows an upward trend in price, with a cumulative return exceeding 20%. Simultaneously, the daily return exhibits high volatility, highlighting the rapid changes and uncertainty of the ETH digital asset. Volatility shows periods of increased risk, particularly during rapid price changes. Furthermore, the Z-score detects abnormal price movements, which may be related to external factors or investor behavior in the market.

While the analysis provides insightful information about price behavior, it has some limitations. For example, the data used is relatively short and may not fully reflect long-term trends, and the statistical methods employed are basic and may not fully explain the complex market behavior. Additionally, detected anomalies may not always be caused by significant factors but may stem from inherent market volatility. Nevertheless, the proposed approach can serve as a foundation for developing higher-level analyses.

7 Conclusions

This research presents a methodology for analyzing Ethereum price data using a custom-built data pipeline and statistical methods. The results reveal price trends, volatility, and anomalies in the market, which can be used to support investment decision-making. The developed system demonstrates the potential of integrating data engineering and data analysis to create a powerful data analysis tool.

8 Future Research

In the future, this research can be further developed in several ways. Machine learning techniques can be applied to price forecasting, and the research can be developed to support real-time data for timely market monitoring. The analysis can be extended to other digital assets for comparative study and investment analysis, and the findings could potentially be applied to investments in stocks or mutual funds.

References

- [1] CoinGecko. Accessed: 2026-04-27.
- [2] Jirapat Klaokliang. Learn databricks, one of the most powerful tools!, 2021. Accessed: 2026-05-01.
- [3] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008.
- [4] Scott Nevil. Z-score: Meaning and formula, 2026-03-15. Accessed: 2026-04-28.
- [5] Ruey S. Tsay. *Analysis of Financial Time Series*. Wiley, 3 edition, 2010.
- [6] Andrew Urquhart. The inefficiency of bitcoin. *Economics Letters*, 148:80–82, 2016.
- [7] Waranyu Suknantee. Get to know ethereum, the world’s second-largest cryptocurrency., 1-02-2021. Accessed: 2026-04-28.

Appendix A: System Development Source Code

Section 1: Data Retrieval and Error Handling via API (Python)

```
1 import requests
2 import pandas as pd
3 import sqlite3
4 import logging
5 from datetime import datetime
6
7 # Set up basic logging
8 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
9 logger = logging.getLogger(__name__)
10
11 # --- IMPROVED: Wrap API call in try-except ---
12 URL = "https://api.coingecko.com/api/v3/coins/ethereum/market_chart"
13 PARAMS = {"vs_currency": "usd", "days": 30, "interval": "daily"}
14
15 try:
16     response = requests.get(URL, params=PARAMS, timeout=10)
17     response.raise_for_status()
18     data = response.json()
19     logger.info("API call successful. Records received.")
20
21 except requests.exceptions.Timeout:
22     logger.error("API request timed out. CoinGecko may be slow.")
23     raise
24 except requests.exceptions.HTTPError as e:
25     logger.error(f"HTTP error from API: {e.response.status_code} - {e}")
26     raise
27 except requests.exceptions.RequestException as e:
28     logger.error(f"Unexpected API error: {e}")
29     raise
```

Part 2: Data Transformation with PySpark (Python))

```
1 from pyspark.sql import functions as F
2 from pyspark.sql.types import StructType, StructField, LongType, DoubleType
3
4 prices = data['prices']
5 volumes = data['total_volumes']
6
7 rows = [
8     (int(p[0]), float(p[1]), float(v[1]))
9     for p, v in zip(prices, volumes)
10 ]
11
12 # Define schema explicitly
13 schema = StructType([
14     StructField("timestamp_ms", LongType(), nullable=False),
15     StructField("price", DoubleType(), nullable=False),
16     StructField("volume", DoubleType(), nullable=False),
17 ])
18
19 df_spark = spark.createDataFrame(rows, schema=schema)
20
21 # Transform: convert timestamp and round values
22 df_clean = df_spark.withColumn(
23     "date", F.to_date(F.from_unixtime(F.col("timestamp_ms") / 1000))
24 ).select(
25     F.col("date"),
26     F.round("price", 2).alias("price"),
27     F.round("volume", 2).alias("volume")
28 )
29
30 df_clean.createOrReplaceTempView("eth_daily")
```

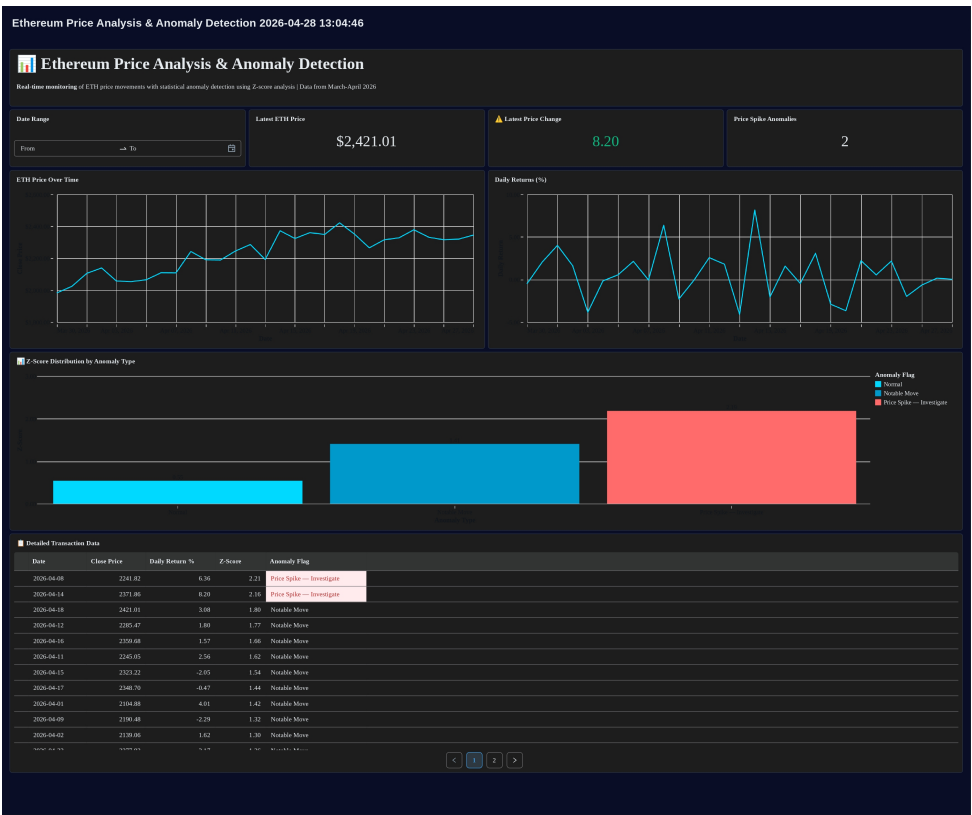
Part 3: Cumulative Return Assessment (Spark SQL))

```
1 WITH daily_returns AS (  
2     SELECT  
3         date, price,  
4         LAG(price) OVER (ORDER BY date) AS prev_price,  
5         ROUND((price - LAG(price) OVER (ORDER BY date))  
6             / LAG(price) OVER (ORDER BY date) * 100, 2) AS daily_return_pct  
7     FROM eth_daily  
8 )  
9 SELECT  
10    date,  
11    ROUND(price, 2) AS close_price,  
12    ROUND(prev_price, 2) AS prev_close,  
13    daily_return_pct,  
14    ROUND((price - FIRST_VALUE(price) OVER (ORDER BY date))  
15        / FIRST_VALUE(price) OVER (ORDER BY date) * 100, 2) AS cumulative_return_pct,  
16    CASE  
17        WHEN daily_return_pct > 3 THEN 'Strong Up'  
18        WHEN daily_return_pct > 0 THEN 'Up'  
19        WHEN daily_return_pct = 0 THEN 'Flat'  
20        WHEN daily_return_pct > -3 THEN 'Down'  
21        ELSE 'Strong Down'  
22    END AS day_sentiment  
23 FROM daily_returns  
24 ORDER BY date DESC;
```

Part 4: Anomaly Detection with Z-Score (Spark SQL)

```
1 WITH stats AS (  
2     SELECT  
3         date, price,  
4         ROUND((price - LAG(price) OVER (ORDER BY date))  
5             / LAG(price) OVER (ORDER BY date) * 100, 4) AS daily_return_pct,  
6         AVG(price) OVER (ORDER BY date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW) AS  
7         rolling_mean_30d,  
8         STDDEV(price) OVER (ORDER BY date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW) AS  
9         rolling_std_30d  
10    FROM eth_daily  
11 ),  
12 zscore_calc AS (  
13     SELECT *,  
14     ROUND((price - rolling_mean_30d) / NULLIF(rolling_std_30d, 0), 2) AS z_score  
15 FROM stats  
16 )  
17 SELECT  
18    date,  
19    ROUND(price, 2) AS close_price,  
20    daily_return_pct,  
21    ROUND(rolling_mean_30d, 2) AS benchmark_avg,  
22    ROUND(rolling_std_30d, 2) AS benchmark_std,  
23    z_score,  
24    CASE  
25        WHEN z_score > 2 THEN 'Price Spike - Investigate'  
26        WHEN z_score < -2 THEN 'Price Crash - Investigate'  
27        WHEN ABS(z_score) BETWEEN 1 AND 2 THEN 'Notable Move'  
28        ELSE 'Normal'  
29    END AS anomaly_flag  
30 FROM zscore_calc  
31 WHERE z_score IS NOT NULL  
32 ORDER BY ABS(z_score) DESC;
```


Appendix B: Example of a summary report.



An example of an Ethereum price analysis report using Databricks.